DÉPLOIEMENT D'APPLICATIONS INTELLIGENTES · MLOps

TP 8 — Appliquer des techniques XAI

Durée : 4 h · Partie 6 du cours · Windows PowerShell

Ce TP fait refaire, sur votre propre poste, chacun des exemples de la partie 6 du cours : le paiement bloqué de Mme Benali, l'importance par permutation, les courbes PDP/ICE, SHAP, LIME et sa stabilité, les contrefactuels, la fuite de données démasquée, les modèles interprétables (logistique, arbre, EBM, monotonie, substitut), l'équité par groupe et la fiche modèle. Tous les outils sont open source. La partie B pose ensuite les mêmes questions avec les outils des TP 5 à 7 : Orange sans code, la plateforme H2O, et un garde-fou d'explicabilité dans un workflow Prefect. Pour chaque étape, vos chiffres doivent retrouver ceux du cours : ils servent de points de contrôle.

VERSION WINDOWS

Toutes les commandes sont écrites pour Windows PowerShell (5.1 ou 7), jamais l'invite cmd.exe. Les blocs `@' ... '@` écrivent un fichier ou l'envoient à Python : le marqueur de fermeture `'@` doit être en tout début de ligne, et le bloc se colle en entier. Les fichiers sont écrits en `-Encoding ascii` : l'encodage UTF8 de PowerShell 5.1 ajoute un marqueur invisible en tête de fichier qui casse la lecture des fichiers YAML et JSON.
Si l'activation de l'environnement est refusée : `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned`, puis relancez `.venv\Scripts\Activate.ps1`.

CE QUI CHANGE DANS CETTE VERSION

Le TP suit désormais la partie 6 exemple par exemple, avec le dossier de Mme Benali (1 800 MAD, de nuit, 30 mois d'ancienneté) au lieu de l'ancien cas à 12 500 MAD. Toutes les sorties ont été obtenues réellement. Nouveau : la partie B (Orange, H2O, Prefect). Durée portée à 4 h ; si le temps manque, la partie B peut être terminée en autonomie.

Objectifs pédagogiques

- Expliquer une décision individuelle avec SHAP et la traduire en une phrase pour le client
- Produire des explications globales (permutation, SHAP, PDP/ICE) et les confronter au métier
- Mesurer la stabilité de LIME et formuler un contrefactuel actionnable
- Démasquer une fuite de données grâce à une explication
- Construire des modèles interprétables et mesurer ce qu'ils coûtent (ou non) en performance
- Mesurer l'équité par groupe avec Fairlearn et rédiger la fiche modèle
- Obtenir les mêmes explications sans code (Orange), sur une plateforme AutoML (H2O) et automatiquement dans un workflow (Prefect)

Prérequis

- TP 3 terminé : `models/model.pkl` (forêt, 300 arbres, profondeur 12, F1 0,8636) ; à défaut, `dvc repro` le recrée
- Serveur MLflow démarré (`mlflow server --host 127.0.0.1 --port 5000`)
- Partie B : l'environnement Orange du TP 5, H2O et Java 17 du TP 6, Prefect du TP 7

Installation

Les bibliothèques d'explicabilité vont dans l'environnement du projet. L'add-on « Explain » d'Orange va dans l'environnement Orange du TP 5 (ou par le menu d'Orange : Options > Add-ons > Explain).

terminal

```
cd tp-mlops; .venv\Scripts\Activate.ps1
pip install shap lime interpret fairlearn matplotlib
python -c "import shap, lime, interpret, fairlearn; print(shap.__version__)"
& $HOME\orange-env\Scripts\pip install orange3-explain   # add-on Explain (partie B)
```

RÉSULTAT ATTENDU

```
0.54.1        (ou plus recent)
Successfully installed ... Orange3-Explain-0.6.11 ...
```

Déroulement

Étape 1 — Le dossier de Mme Benali et l'importance globale (15 min)

Partie A : les exemples du cours avec les bibliothèques Python. Un petit module commun charge le modèle, les données, le découpage habituel et le dossier de la cliente du cours. Puis on compare l'importance « MDI » fournie par la forêt et l'importance par permutation.

src/xai_commun.py (module commun, importé par tous les scripts)

```
@'
import joblib, pandas as pd
from sklearn.model_selection import train_test_split
m = joblib.load('models/model.pkl')                      # foret du TP 2/3 (300 arbres)
df = pd.read_csv('data/transactions.csv')
X, y = df.drop(columns='fraude'), df['fraude']
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42)   # meme decoupage
CAS = pd.DataFrame([{'montant': 1800.0, 'anciennete': 30, 'freq_24h': 4, 'nuit': 1}])  # Mme Benali
'@ | Set-Content -Encoding ascii src/xai_commun.py
```

src/xai_importance.py

```
@'
import pandas as pd
from sklearn.inspection import permutation_importance
from xai_commun import m, X, Xte, yte, CAS
print('P(fraude) Mme Benali :', round(m.predict_proba(CAS)[0, 1], 4))
r = permutation_importance(m, Xte, yte, scoring='f1', n_repeats=10, random_state=0)
t = pd.DataFrame({'MDI (foret)': m.feature_importances_, 'permutation': r.importances_mean,
                  'ecart-type': r.importances_std}, index=X.columns).round(3)
print(t.sort_values('permutation', ascending=False).to_string())
'@ | Set-Content -Encoding ascii src/xai_importance.py
python src/xai_importance.py
```

RÉSULTAT ATTENDU

```
P(fraude) Mme Benali : 0.8899
            MDI (foret)  permutation  ecart-type
montant           0.480        0.591       0.032
freq_24h          0.191        0.265       0.022
anciennete        0.236        0.263       0.029
nuit              0.093        0.193       0.016
```

Si ça coince : lancez toujours les scripts depuis le dossier `tp-mlops` (`python src/...`) : les chemins `models/` et `data/` sont relatifs à ce dossier.

Étape 2 — PDP et ICE : la forme de l'effet (10 min)

src/xai_pdp.py

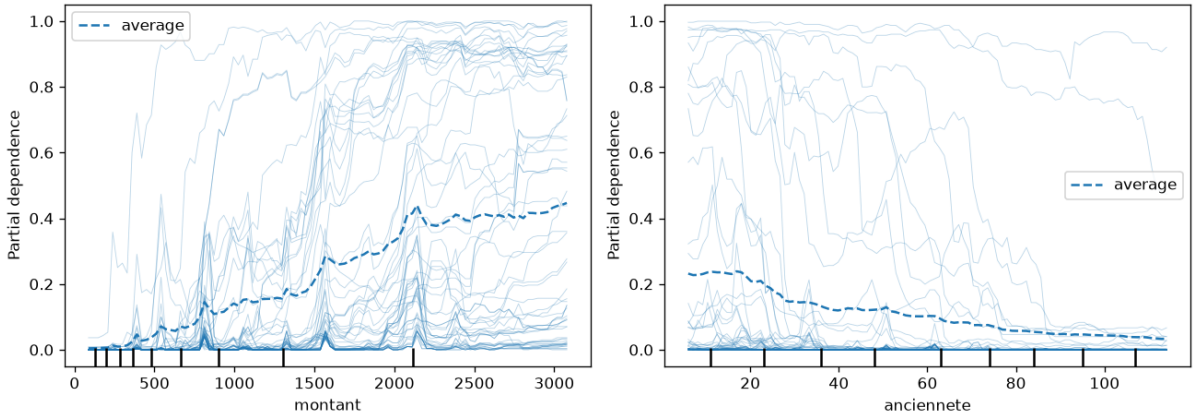
```
@'
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.inspection import PartialDependenceDisplay, partial_dependence
from xai_commun import m, Xte
Xf = Xte.astype(float)                                  # le PDP refuse les colonnes entieres
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
PartialDependenceDisplay.from_estimator(m, Xf, ['montant', 'anciennete'], kind='both',
                                        subsample=60, random_state=0, ax=ax)
plt.tight_layout(); plt.savefig('reports/pdp_ice.png', dpi=120)
for v in ['montant', 'anciennete']:
    moy = partial_dependence(m, Xf, [v], kind='average')['average'][0]
    print(f'{v:11s} PDP de {moy.min():.3f} a {moy.max():.3f}')
'@ | Set-Content -Encoding ascii src/xai_pdp.py
```

```
python src/xai_pdp.py
```

RÉSULTAT ATTENDU

```
montant     PDP de 0.004 a 0.447
anciennete  PDP de 0.033 a 0.239

reports/pdp_ice.png : une courbe moyenne en pointilles
(PDP) et 60 courbes individuelles (ICE) par variable.
```



Sortie réelle : reports/pdp_ice.png.

Si ça coince : « Partial dependence plots are not supported for integer data » : c'est pour cela que le script convertit les colonnes en réels (`astype(float)`).

Étape 3 — SHAP local : pourquoi ce paiement a été bloqué (15 min)

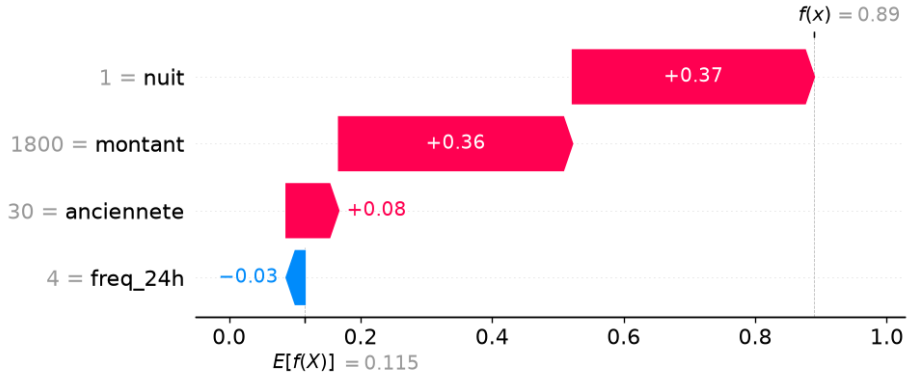
C'est l'explication exigée quand la cliente demande pourquoi. On vérifie aussi la propriété d'efficacité : moyenne du modèle + somme des contributions = prédiction.

src/xai_local.py

```
@'
import shap
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from xai_commun import m, CAS
ex = shap.TreeExplainer(m)                              # exact et rapide pour les arbres
e = ex(CAS)[0, :, 1]                                    # classe 1 = fraude
print('moyenne du modele :', round(e.base_values, 4))
for nom, c in sorted(zip(CAS.columns, e.values), key=lambda t: -abs(t[1])):
    print(f'{nom:11s} {c:+.4f}')
print('somme            :', round(e.base_values + e.values.sum(), 4))
print('prediction       :', round(m.predict_proba(CAS)[0, 1], 4))
shap.plots.waterfall(e, show=False)
plt.tight_layout(); plt.savefig('reports/shap_benali.png', dpi=120)
'@ | Set-Content -Encoding ascii src/xai_local.py
python src/xai_local.py
```

RÉSULTAT ATTENDU

```
moyenne du modele : 0.1153
nuit        +0.3677
montant     +0.3555
anciennete  +0.0798
freq_24h    -0.0284
somme             : 0.8899
prediction        : 0.8899
```



Sortie réelle : reports/shap_benali.png (waterfall).

Rédigez la phrase que vous enverriez à la cliente, en une ligne et sans jargon. Comparez avec celle du cours (diapositive 26).

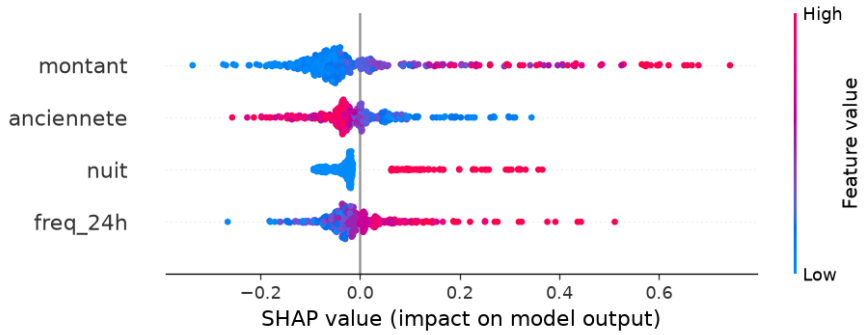
Étape 4 — SHAP global et dépendance (10 min)

src/xai_global.py

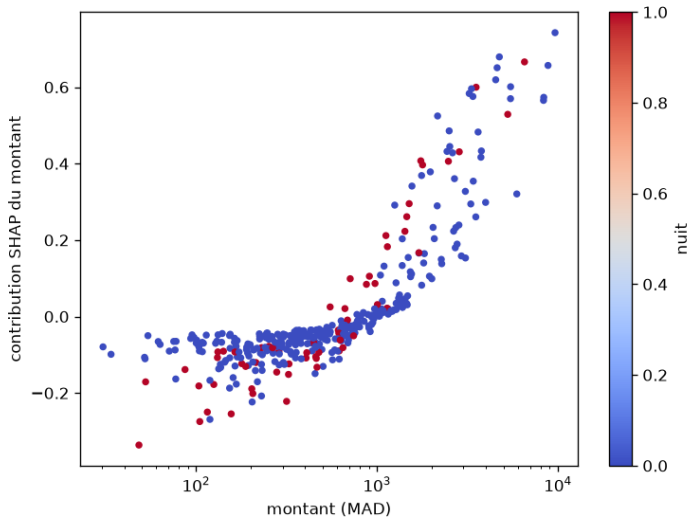
```
@'
import numpy as np, pandas as pd, shap
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from xai_commun import m, Xte
Xs = Xte.sample(400, random_state=0)
sv = shap.TreeExplainer(m)(Xs)[:, :, 1]                # 400 explications, classe fraude
imp = pd.Series(np.abs(sv.values).mean(0), index=Xs.columns)
print(imp.sort_values(ascending=False).round(4).to_string())
shap.plots.beeswarm(sv, show=False)
plt.tight_layout(); plt.savefig('reports/shap_global.png', dpi=120); plt.close('all')
plt.scatter(Xs['montant'], sv[:, 'montant'].values, c=Xs['nuit'], cmap='coolwarm', s=12)
plt.xscale('log'); plt.xlabel('montant (MAD)'); plt.ylabel('contribution SHAP du montant')
plt.colorbar(label='nuit'); plt.tight_layout(); plt.savefig('reports/shap_montant.png', dpi=120)
print('graphiques ecrits')
'@ | Set-Content -Encoding ascii src/xai_global.py
python src/xai_global.py
```

RÉSULTAT ATTENDU

```
montant       0.1155
ancienne te    0.0578
nuit          0.0572
freq_24h      0.0555
graphiques ecrits
```



Sortie réelle : reports/shap_global.png (beeswarm).



Sortie réelle : reports/shap_montant.png. Le seuil se situe vers 1 000 MAD.

Étape 5 — LIME et la stabilité de l'explication (15 min)

Même dossier, cinq graines, deux tailles de voisinage. Notez quelles contributions changent de signe.

src/xai_lime.py

```
@'
from lime.lime_tabular import LimeTabularExplainer
from xai_commun import m, X, Xtr, CAS
ex0 = LimeTabularExplainer(Xtr.values, feature_names=list(X.columns), mode='classification',
                           class_names=['legitime', 'fraude'], random_state=0)
print('regles :', [k for k, _ in ex0.explain_instance(CAS.values[0], m.predict_proba,
                                                      num_features=4).as_list()])
for n in [500, 2000]:                                  # taille du voisinage
    print(f'--- {n} voisins      montant    nuit   anciennete  freq_24h')
    for graine in range(5):                            # 5 tirages : l'explication est-elle stable ?
        ex = LimeTabularExplainer(Xtr.values, feature_names=list(X.columns), mode='classification',
                                  class_names=['legitime', 'fraude'], random_state=graine)
        e = dict(ex.explain_instance(CAS.values[0], m.predict_proba, num_features=4,
                                     num_samples=n).as_list())
        w = [next(v for k, v in e.items() if col in k) for col in X.columns]
        print(f'graine {graine}         {w[0]:+.3f}   {w[3]:+.3f}   {w[1]:+.3f}     {w[2]:+.3f}')
'@ | Set-Content -Encoding ascii src/xai_lime.py
python src/xai_lime.py
```

RÉSULTAT ATTENDU

```
regles : ['montant > 1007.36', 'nuit > 0.00', '29.00 < anciennete <= 60.00', '3.00 < freq_24h <= 4.00']
--- 500 voisins      montant    nuit   anciennete  freq_24h
graine 0         +0.459   +0.244   -0.007     -0.035
graine 1         +0.499   +0.245   +0.060     -0.018
graine 2         +0.406   +0.208   +0.002     -0.021
graine 3         +0.384   +0.192   +0.015     -0.028
graine 4         +0.479   +0.189   -0.005     -0.035
--- 2000 voisins      montant    nuit   anciennete  freq_24h
graine 0         +0.483   +0.182   +0.026     +0.001
graine 1         +0.429   +0.236   +0.032     -0.037
graine 2         +0.440   +0.180   +0.025     +0.012
graine 3         +0.455   +0.205   +0.020     -0.016
graine 4         +0.455   +0.228   +0.034     -0.003
```

CE QU'IL FAUT VOIR

Montant et nuit restent stables. Avec 500 voisins, l'ancienneté change de signe selon la graine (–0,007 / +0,060) ; avec 2 000 voisins, c'est encore la fréquence (–0,037 / +0,012). On ne communique à la cliente que ce qui est stable.

Étape 6 — Contrefactuels : qu'aurait-il fallu ? (10 min)

src/contrefactuel.py

```
@'
import numpy as np
from xai_commun import m, CAS
p = lambda d: m.predict_proba(d)[0, 1]
print('situation reelle       : P =', round(p(CAS), 3))
c = CAS.copy(); c['nuit'] = 0
print('meme paiement de jour  : P =', round(p(c), 3))
c = CAS.copy()
for v in np.arange(1800, 0, -10):                       # on baisse le montant pas a pas
    c['montant'] = v
    if p(c) < 0.5: print(f'montant <= {v} MAD     : P =', round(p(c), 3)); break
c = CAS.copy()
for a in range(30, 121):                                # on augmente l'anciennete
    c['anciennete'] = a
    if p(c) < 0.5: print(f'anciennete >= {a} mois  : P =', round(p(c), 3)); break
'@ | Set-Content -Encoding ascii src/contrefactuel.py
python src/contrefactuel.py
```

RÉSULTAT ATTENDU

```
situation reelle       : P = 0.89
meme paiement de jour  : P = 0.098
montant <= 530 MAD     : P = 0.415
anciennete >= 92 mois  : P = 0.397

Trois reponses : laquelle est actionnable par la cliente ? laquelle ne l'est pas ?
```

Étape 7 — Démasquer une fuite de données (10 min)

On ajoute une colonne `statut_enquete`, connue seulement après l'enquête de la banque. Le score s'envole ; l'importance par permutation dénonce la colonne. Le script enregistre aussi ce modèle « tricheur », réutilisé à l'étape 14.

src/fuite.py

```
@'
import joblib, numpy as np, pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import f1_score
from sklearn.model_selection import train_test_split
df = pd.read_csv('data/transactions.csv')
rng = np.random.default_rng(1)                          # resultat de l'enquete, connu APRES coup
df['statut_enquete'] = np.where(rng.random(len(df)) < 0.97, df['fraude'], 1 - df['fraude'])
X, y = df.drop(columns='fraude'), df['fraude']
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42)
m = RandomForestClassifier(n_estimators=300, max_depth=12, random_state=42).fit(Xtr, ytr)
print('F1 avec la colonne :', round(f1_score(yte, m.predict(Xte)), 4))
r = permutation_importance(m, Xte, yte, scoring='f1', n_repeats=5, random_state=0)
print(pd.Series(r.importances_mean, index=X.columns).sort_values(ascending=False).round(3).to_string())
joblib.dump(m, 'models/model_fuite.pkl'); df.to_csv('data/transactions_fuite.csv', index=False)
print('modele et donnees avec fuite ecrits')
'@ | Set-Content -Encoding ascii src/fuite.py
python src/fuite.py
```

RÉSULTAT ATTENDU

```
F1 avec la colonne : 0.9323
statut_enquete    0.754
montant           0.285
nuit              0.056
anciennete        0.046
freq_24h          0.045
modele et donnees avec fuite ecrits
```

Étape 8 — Modèles lisibles : logistique et arbre (15 min)

Section 3 du cours. On mesure ce que coûte la lisibilité, et l'effet d'une seule variable bien construite : le logarithme du montant.

src/lisibles.py

```
@'
import numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, export_text
from xai_commun import m, X, Xtr, Xte, ytr, yte, CAS
def logm(d):                                          # le montant agit par ordre de grandeur
    d = d.copy(); d['log_montant'] = np.log10(d.pop('montant')); return d
for nom, f in [('montant brut', lambda d: d), ('log(montant)', logm)]:
    lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(f(Xtr), ytr)
    print(f'logistique, {nom:12s} F1 = {f1_score(yte, lr.predict(f(Xte))):.4f}')
coef = pd.Series(lr[-1].coef_[0], index=logm(Xtr).columns)
print('coefficients standardises :', coef.round(2).to_dict())
brut = np.exp(coef / lr[0].scale_)                    # effet d'une unite, en multiplicateur de cote
print('cote x', round(brut['anciennete'], 3), 'par mois d\'anciennete ; x',
      round(brut['freq_24h'], 2), 'par transaction en 24 h')
for p in [2, 3, 4, 6, None]:
    t = DecisionTreeClassifier(max_depth=p, random_state=0).fit(Xtr, ytr)
    print(f'arbre profondeur {str(p):4s}: {t.get_n_leaves():3d} feuilles, F1 = '
          f'{f1_score(yte, t.predict(Xte)):.4f}')
t3 = DecisionTreeClassifier(max_depth=3, random_state=0).fit(Xtr, ytr)
print(export_text(t3, feature_names=list(X.columns), decimals=1))
print('Mme Benali : feuille', t3.apply(CAS)[0], '| P =', round(t3.predict_proba(CAS)[0, 1], 3))
print('foret     : F1 =', round(f1_score(yte, m.predict(Xte)), 4))
'@ | Set-Content -Encoding ascii src/lisibles.py
python src/lisibles.py
```

RÉSULTAT ATTENDU

```
logistique, montant brut F1 = 0.7407
logistique, log(montant) F1 = 0.8593
coefficients standardises : {'anciennete': -2.58, 'freq_24h': 2.75, 'nuit': 2.23, 'log_montant': 4.93}
cote x 0.928 par mois d'anciennete ; x 4.76 par transaction en 24 h
arbre profondeur 2   :   4 feuilles, F1 = 0.2293
arbre profondeur 3   :   8 feuilles, F1 = 0.5737
arbre profondeur 4   :  16 feuilles, F1 = 0.6063
arbre profondeur 6   :  50 feuilles, F1 = 0.7510
arbre profondeur None: 206 feuilles, F1 = 0.7556
|--- montant <= 1505.7
|   |--- freq_24h <= 6.5
|   |   |--- nuit <= 0.5
|   |   |   |--- class: 0
|   |   |--- nuit >  0.5
|   |   |   |--- class: 0
|   |--- freq_24h >  6.5
|   |   |--- montant <= 371.9
|   |   |   |--- class: 0
|   |   |--- montant >  371.9
|   |   |   |--- class: 0
|--- montant >  1505.7
|   |--- nuit <= 0.5
|   |   |--- anciennete <= 69.5
|   |   |   |--- class: 1
|   |   |--- anciennete >  69.5
|   |   |   |--- class: 0
|   |--- nuit >  0.5
|   |   |--- anciennete <= 64.0
|   |   |   |--- class: 1
|   |   |--- anciennete >  64.0
|   |   |   |--- class: 1

Mme Benali : feuille 13 | P = 0.983
foret     : F1 = 0.8636
```

Réécrivez l'arbre de profondeur 3 en trois règles (« SI ... ALORS fraude »), puis suivez le chemin de Mme Benali jusqu'à la feuille 13.

Si ça coince : à quelques millièmes près, l'arbre de profondeur 6 peut donner 0,7510 ou 0,7529 selon les versions de bibliothèques : les égalités entre seuils ne sont pas départagées partout de la même façon.

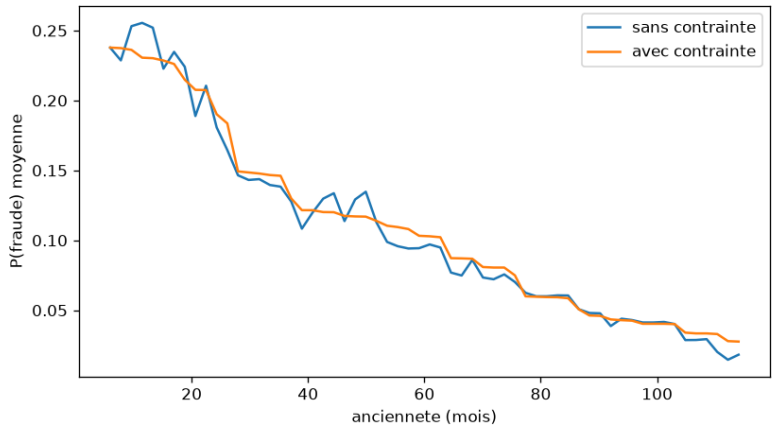
Étape 9 — EBM, contraintes de monotonie, modèle de substitution (15 min)

src/ebm_monotonie.py

```
@'
import numpy as np
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from interpret.glassbox import ExplainableBoostingClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.inspection import partial_dependence
from sklearn.metrics import f1_score
from sklearn.tree import DecisionTreeClassifier
from xai_commun import m, Xtr, Xte, ytr, yte, CAS
# 1. EBM : une courbe par variable, explication exacte
ebm = ExplainableBoostingClassifier(interactions=0, random_state=42).fit(Xtr, ytr)
print('EBM           F1 =', round(f1_score(yte, ebm.predict(Xte)), 4))
loc = ebm.explain_local(CAS).data(0)
print('Mme Benali     :', {k: round(float(v), 2) for k, v in zip(loc['names'], loc['scores'])},
      '+ constante', round(ebm.intercept_[0], 2), '-> P =', round(ebm.predict_proba(CAS)[0, 1], 3))
# 2. Boosting avec et sans contrainte de monotonie (montant +, anciennete -, freq +, nuit +)
fig, ax = plt.subplots(figsize=(7, 4))
for cst, lab in [(None, 'sans contrainte'), ([1, -1, 1, 1], 'avec contrainte')]:
    h = HistGradientBoostingClassifier(random_state=0, monotonic_cst=cst).fit(Xtr, ytr)
    print(f'boosting {lab:16s} F1 = {f1_score(yte, h.predict(Xte)):.4f}')
    pd_ = partial_dependence(h, Xte.astype(float), ['anciennete'], method='brute',
                             response_method='predict_proba', grid_resolution=60)
    ax.plot(pd_['grid_values'][0], pd_['average'][0], label=lab)
ax.set_xlabel('anciennete (mois)'); ax.set_ylabel('P(fraude) moyenne'); ax.legend()
plt.tight_layout(); plt.savefig('reports/monotonie.png', dpi=120)
# 3. Substitut : un arbre court qui imite la foret
pf_tr, pf_te = m.predict(Xtr), m.predict(Xte)
for p in [3, 5]:
    s = DecisionTreeClassifier(max_depth=p, random_state=0).fit(Xtr, pf_tr)
    acc = (s.predict(Xte) == pf_te).mean(); accf = (s.predict(Xte)[pf_te == 1] == 1).mean()
    print(f'substitut prof. {p} : accord {acc:.1%}, accord sur les fraudes de la foret {accf:.1%}')
'@ | Set-Content -Encoding ascii src/ebm_monotonie.py
python src/ebm_monotonie.py
```

RÉSULTAT ATTENDU

```
EBM           F1 = 0.8538
Mme Benali     : {'montant': 5.69, 'anciennete': 1.74, 'freq_24h': 0.04, 'nuit': 4.37} + constante -7.7
3 -> P = 0.984
boosting sans contrainte  F1 = 0.8390
boosting avec contrainte  F1 = 0.8527
substitut prof. 3 : accord 92.1%, accord sur les fraudes de la foret 56.9%
substitut prof. 5 : accord 95.4%, accord sur les fraudes de la foret 74.6%
```



Sortie réelle : reports/monotonie.png. Sans contrainte, le risque remonte vers 45 mois.

Vérifiez à la main l'explication de l'EBM : –7,73 + 5,69 + 1,74 + 0,04 + 4,37 = 4,11, et 1 / (1 + e^–4,11) = 0,984. Pour le substitut, expliquez pourquoi 92 % d'accord global cachent seulement 57 % d'accord sur les fraudes.

Étape 10 — Mesurer l'équité avec Fairlearn (10 min)

Un groupe A/B fictif, plus fréquent chez les clients récents, est attribué aux transactions de test. Le modèle ne le voit jamais.

src/equite.py

```
@'
import numpy as np, pandas as pd
from fairlearn.metrics import MetricFrame, false_positive_rate, selection_rate
from sklearn.metrics import recall_score
from xai_commun import m, Xte, yte
rng = np.random.default_rng(7)                          # groupe A/B FICTIF, pour l'exercice
p_b = np.where(Xte['anciennete'] < 24, 0.75, 0.25)      # B plus frequent chez les clients recents
groupe = pd.Series(np.where(rng.random(len(Xte)) < p_b, 'B', 'A'), index=Xte.index, name='groupe')
pred = m.predict(Xte)                                   # le modele ne voit JAMAIS le groupe
mf = MetricFrame(metrics={'taux_alerte': selection_rate, 'fausses_alertes': false_positive_rate,
                          'rappel': recall_score}, y_true=yte, y_pred=pred, sensitive_features=groupe)
print(mf.by_group.round(4).to_string())
print('taux reel de fraude :', yte.groupby(groupe).mean().round(4).to_dict())
print('ecart entre groupes :', mf.difference().round(4).to_dict())
'@ | Set-Content -Encoding ascii src/equite.py
python src/equite.py
```

RÉSULTAT ATTENDU

```
        taux_alerte  fausses_alertes  rappel
groupe
A            0.0783           0.0108  0.8462
B            0.1506           0.0213  0.8551
taux reel de fraude : {'A': 0.0807, 'B': 0.1551}
ecart entre groupes : {'taux_alerte': 0.0723, 'fausses_alertes': 0.0105, 'rappel': 0.0089}

Les clients honnetes du groupe B subissent deux fois plus
de fausses alertes (2,13 % contre 1,08 %).
```

Étape 11 — Journaliser les explications et rédiger la fiche modèle (15 min)

Les explications doivent vivre avec la version du modèle, pas dans un dossier oublié. La fiche modèle est préremplie avec les valeurs du TP : complétez la rubrique « Limites connues ».

MODEL_CARD.md

```
@'
# Fiche modele - detection-fraude (foret du TP 3)
## Usage prevu
Signaler une transaction pour verification humaine. Le client recoit une raison et un contrefactuel.
## Usages exclus
Blocage definitif sans revue humaine (loi 09-08, art. 11). Octroi de credit.
## Donnees
transactions.csv, 5 000 lignes, 11,4 % de fraudes, version DVC du TP 4.
## Performances (test, 1 250 transactions)
F1 0.8636 - precision 0.8769 - rappel 0.8507.
## Explicabilite
SHAP (TreeExplainer) par decision ; graphiques globaux journalises dans MLflow.
Variables les plus influentes : montant, puis anciennete, nuit, freq_24h.
## Equite (groupe A/B fictif du TP)
Fausses alertes : A 1,08 % ; B 2,13 %. Variable proxy identifiee : anciennete.
## Alternatives interpretables mesurees
Logistique + log(montant) : F1 0.8593. EBM : F1 0.8538.
## Limites connues
A COMPLETER par le binome.
## Revue
Auteurs : ... - revue prevue dans 6 mois.
'@ | Set-Content -Encoding ascii MODEL_CARD.md
```

src/journaliser.py

```
@'
import json, mlflow
mlflow.set_tracking_uri('http://127.0.0.1:5000')
mlflow.set_experiment('tp8-explicabilite')
with mlflow.start_run(run_name='xai-foret-v1'):
    for f in ['shap_global.png', 'shap_benali.png', 'shap_montant.png', 'pdp_ice.png', 'monotonie.png']:
        mlflow.log_artifact('reports/' + f, artifact_path='explications')
    mlflow.log_metrics({'f1': json.load(open('reports/metrics.json'))['f1'],
                        'ecart_fausses_alertes': 0.0105})       # l'equite devient une metrique
    mlflow.log_artifact('MODEL_CARD.md')
    mlflow.set_tag('revue', 'a valider par le metier')
print('explications et fiche modele journalisees')
'@ | Set-Content -Encoding ascii src/journaliser.py
python src/journaliser.py
```

RÉSULTAT ATTENDU

```
explications et fiche modele journalisees

Interface MLflow, experience tp8-explicabilite : onglet Artifacts,
dossier explications/ (5 images) et MODEL_CARD.md.
```

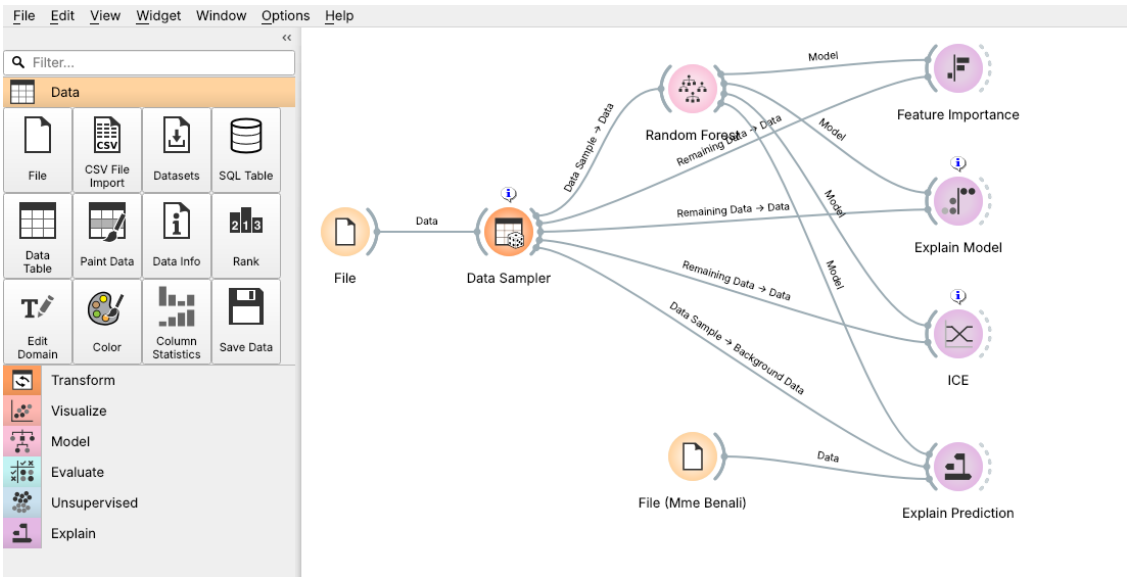
terminal

```
git add src MODEL_CARD.md reports
git commit -m 'xai: explications et fiche modele'
```

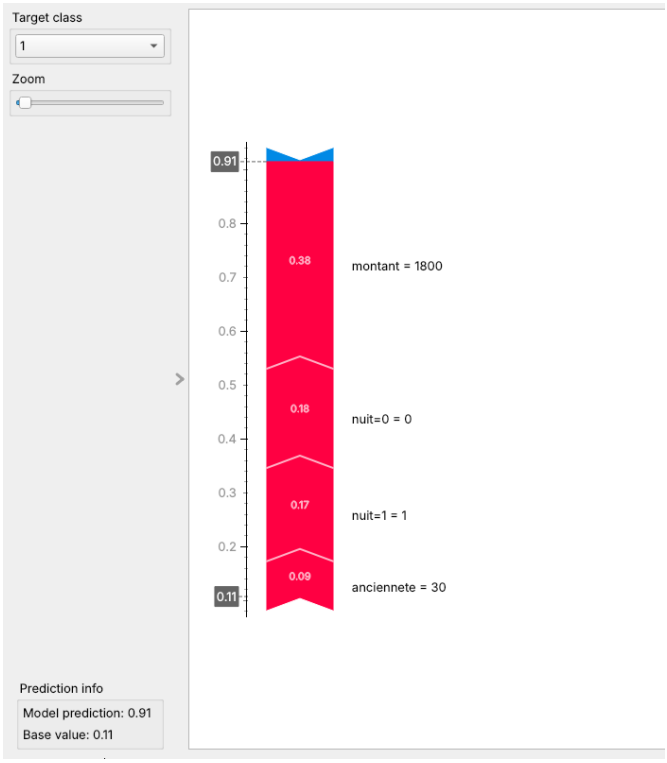
Étape 12 — Partie B · Orange : les mêmes explications sans code (30 min)

Partie B : les outils des TP 5 à 7. L'add-on **Explain** ajoute à Orange une catégorie de widgets qui reposent sur SHAP. On reconstruit le dossier de Mme Benali à la souris. Préparez d'abord un fichier `benali.csv` d'une ligne : `montant,anciennete,freq_24h,nuit` puis `1800.0,30,4,1` (avec un éditeur de texte).

1. Lancez Orange (environnement du TP 5). Vérifiez que la catégorie **Explain** apparaît en bas de la boîte à outils ; sinon, menu **Options > Add-ons**, cochez **Explain**, **OK**, et redémarrez Orange.
2. Comme au TP 5 : **File** sur `transactions.csv` avec `fraude` en **target / categorical**, puis **Data Sampler** (75 %, replicable, stratified).
3. **Random Forest** (catégorie Model) : 100 arbres, **Replicable training** coché. Reliez Data Sampler → Random Forest (**Data Sample** → **Data**) : le widget produit un **Model**.
4. Catégorie **Explain** : posez **Feature Importance**, **Explain Model** et **ICE**. Reliez à chacun le **Model** de Random Forest, et la **Remaining Data** de Data Sampler sur l'entrée **Data**.
5. Posez **Explain Prediction**. Il a trois entrées : **Model** (Random Forest), **Background Data** (Data Sample du Data Sampler) et **Data** : un second widget **File** qui lit `benali.csv`.
6. Dans **Explain Model**, **ICE** et **Explain Prediction**, réglez **Target class** sur **1**. Dans ICE, choisissez la variable **montant**.
7. Relevez : le classement de Feature Importance, la forme du nuage d'Explain Model, la courbe moyenne d'ICE, et pour Mme Benali la prédiction, la valeur de base et les contributions d'Explain Prediction. Enregistrez le workflow (**File > Save As**, `workflows/explication-tp8.ows`).



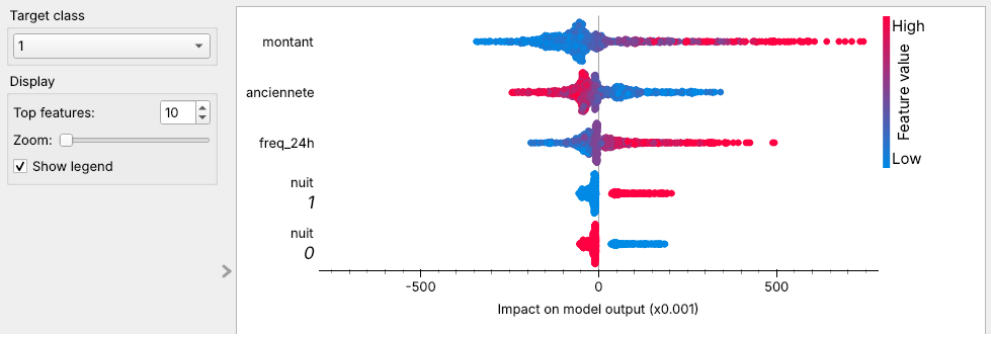
Capture réelle : le workflow de l'étape 12.



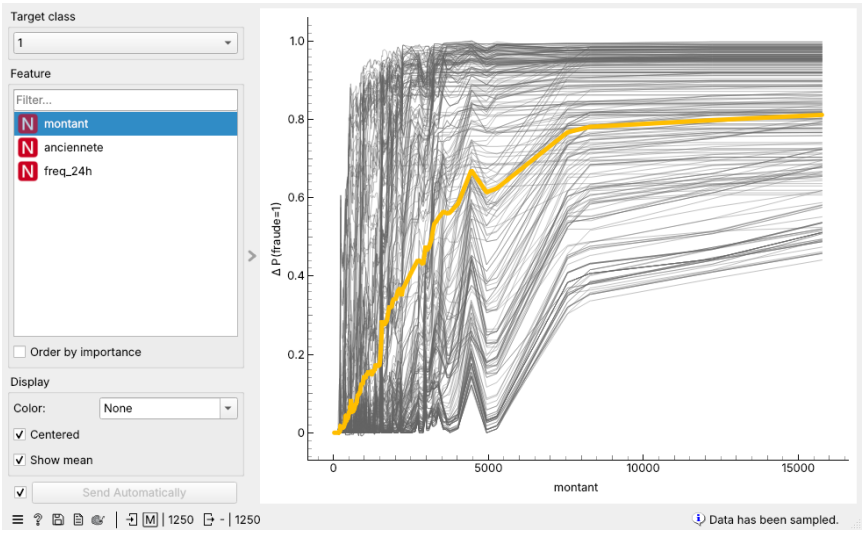
Capture réelle : Explain Prediction pour Mme Benali, classe cible 1.

RÉSULTAT ATTENDU

```
Explain Prediction (Mme Benali) : prediction 0,91 ; valeur de base 0,11
  montant = 1800 : +0,38
  nuit=0 : +0,18   et   nuit=1 : +0,17   (total +0,36)
  anciennete = 30 : +0,09     freq_24h : -0,03
Feature Importance (baisse d'AUC, 5 permutations) :
  montant 0,275 > nuit 0,094 > freq_24h 0,083 > anciennete 0,058
```



Capture réelle : Explain Model, classe cible 1 (même lecture que le beeswarm de l'étape 4).



Capture réelle : ICE sur le montant, classe 1, centré.

DEUX DIFFÉRENCES À EXPLIQUER

1. Orange traite `nuit` comme une variable catégorielle : elle devient deux colonnes (nuit=0, nuit=1) et sa contribution est partagée entre elles (0,18 + 0,17). 2. Ce n'est pas le même modèle qu'en partie A (100 arbres, autre découpage) : 0,91 au lieu de 0,89. Le diagnostic, lui, est le même : montant et nuit.

Si ça coince : si la valeur de base vaut 0,89 et la prédiction 0,09, la classe cible est restée sur **0** : repassez-la à **1**. KNIME (TP 5) propose des équivalents dans son extension « Machine Learning Interpretability » (nœuds SHAP, LIME, PDP/ICE) : à explorer en autonomie, non testés ici.

Étape 13 — Partie B · H2O : expliquer le modèle de la plateforme AutoML (20 min)

Plateforme du TP 6. Les modèles « Stacked Ensemble » ne fournissent pas de contributions SHAP : on explique donc le meilleur modèle XGBoost de la même recherche. H2O renvoie les contributions en log-cote, avec un terme constant (BiasTerm).

terminal 1 : la plateforme (comme au TP 6)

```
cd tp-mlops; .venv\Scripts\Activate.ps1
$JAR = (python -c "import h2o, os; print(os.path.dirname(h2o.__file__))") + "\backend\bin\h2o.jar"
java -jar $JAR -port 54321
```

terminal 2 : src/h2o_xai.py

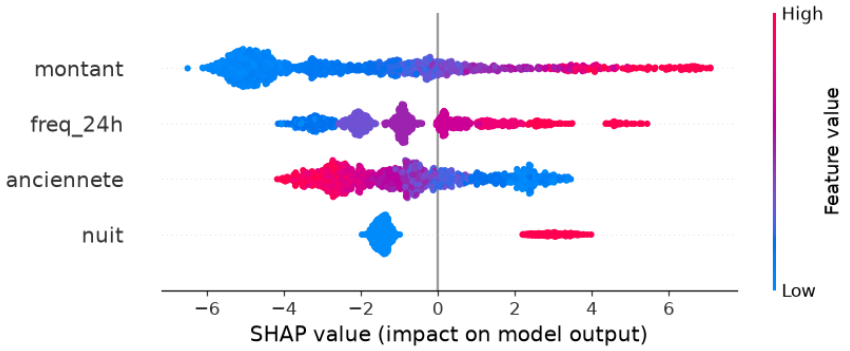
```
@'
import h2o, pandas as pd
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from h2o.automl import H2OAutoML
from sklearn.model_selection import train_test_split
h2o.init(url='http://127.0.0.1:54321')                    # plateforme H2O du TP 6
df = pd.read_csv('data/transactions.csv')
tr, te = train_test_split(df, test_size=0.25, random_state=42)
htr, hte = h2o.H2OFrame(tr), h2o.H2OFrame(te)
for f in (htr, hte): f['fraude'] = f['fraude'].asfactor()
```

```
aml = H2OAutoML(max_models=10, seed=42, exclude_algos=['DeepLearning'], sort_metric='AUC')
aml.train(y='fraude', training_frame=htr)                    # meme recherche qu'au TP 6
xgb = aml.get_best_model(algorithm='xgboost')                # SHAP : modeles a base d'arbres
print('modele explique :', xgb.model_id)
print(xgb.varimp(use_pandas=True)[['variable', 'scaled_importance']].round(3).to_string(index=False))
cas = h2o.H2OFrame(pd.DataFrame([{'montant': 1800.0, 'anciennete': 30, 'freq_24h': 4, 'nuit': 1}]))
contrib = xgb.predict_contributions(cas).as_data_frame().round(3)   # SHAP, en log-cote
print('Mme Benali (log-cote) :', contrib.iloc[0].to_dict())
print('P(fraude) =', round(xgb.predict(cas).as_data_frame()['p1'][0], 3))
import shap                                          # graphique SHAP depuis les contributions H2O
c = xgb.predict_contributions(hte.drop('fraude')).as_data_frame()
X = te.drop(columns='fraude')
e = shap.Explanation(values=c[X.columns].values, base_values=c['BiasTerm'].values, data=X.values,
                     feature_names=list(X.columns))
shap.plots.beeswarm(e, show=False); plt.savefig('reports/h2o_shap.png', dpi=110, bbox_inches='tight')
print('graphique ecrit')
'@ | Set-Content -Encoding ascii src/h2o_xai.py
python src/h2o_xai.py
```

RÉSULTAT ATTENDU

```
modele explique : XGBoost_2_AutoML_2_20261027_135458
  variable  scaled_importance
   montant             1.000
anciennete             0.533
  freq_24h             0.487
      nuit             0.285
Mme Benali (log-cote) : {'montant': 3.023, 'anciennete': 0.456, 'freq_24h': -0.723, 'nuit': 3.384, 'Bia
sTerm': -2.463}
P(fraude) = 0.975
graphique ecrit

Verification : -2,463 + 3,023 + 0,456 - 0,723 + 3,384 = 3,677
et 1 / (1 + e^-3,677) = 0,975.
```



Sortie réelle : reports/h2o_shap.png (contributions H2O).

POURQUOI NE PAS UTILISER xgb.shap_summary_plot() ?

Avec H2O 3.46 et numpy 2, les graphiques intégrés d'H2O échouent (erreur réelle : « 'cols_py' expects 2 but was passed 3 »). On récupère donc les contributions avec `predict_contributions` et on trace le graphique avec SHAP : même information, sans le bogue. Avec numpy 1.x, `xgb.shap_summary_plot(hte)` et `aml.explain(hte)` fonctionnent.

Étape 14 — Partie B · Prefect : un garde-fou d'explicabilité automatique (20 min)

Orchestrateur du TP 7. On ajoute au cycle de réentraînement un contrôle automatique : si une variable inconnue au moment de la décision apparaît dans les importances, ou si une seule variable écrase toutes les autres, le modèle est refusé. On le teste sur le bon modèle, puis sur le modèle « tricheur » de l'étape 7.

src/flow_xai.py

```
@'
import sys, joblib, pandas as pd
from prefect import flow, task, get_run_logger
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
```

```
AUTORISEES = {'montant', 'anciennete', 'freq_24h', 'nuit'}   # connues au moment de la decision
@task
def expliquer(modele, donnees):
    m, df = joblib.load(modele), pd.read_csv(donnees)
    X, y = df.drop(columns='fraude'), df['fraude']
    _, Xte, _, yte = train_test_split(X, y, test_size=0.25, random_state=42)
    r = permutation_importance(m, Xte, yte, scoring='f1', n_repeats=5, random_state=0)
    return pd.Series(r.importances_mean, index=X.columns).sort_values(ascending=False)
@task
def controler(imp):
    log = get_run_logger()
    log.info('importances : ' + ', '.join(f'{k} {v:.3f}' for k, v in imp.items()))
    interdites = set(imp.index) - AUTORISEES
    if interdites:                                  # garde-fou 1 : variable inconnue a la decision
        raise ValueError(f'variables non autorisees : {sorted(interdites)}')
    part = imp.iloc[0] / imp.clip(lower=0).sum()
    if part > 0.6:                                  # garde-fou 2 : une variable ecrase les autres
        raise ValueError(f'{imp.index[0]} represente {part:.0%} de l importance')
    log.info(f'explications plausibles : {imp.index[0]} en tete ({part:.0%})')
@flow(name='controle-explicabilite')
def controle(modele='models/model.pkl', donnees='data/transactions.csv'):
    controler(expliquer(modele, donnees))
if __name__ == '__main__':
    controle(*sys.argv[1:])
'@ | Set-Content -Encoding ascii src/flow_xai.py
```

terminal

```
python src/flow_xai.py                          # le modele du TP 3
python src/flow_xai.py models/model_fuite.pkl data/transactions_fuite.csv
```

RÉSULTAT ATTENDU

```
Premier lancement :
Task run 'expliquer-eca' - Finished in state Completed()
Task run 'controler-dd5' - importances : montant 0.586, freq_24h 0.283, anciennete 0.264, nuit 0.194
Task run 'controler-dd5' - explications plausibles : montant en tete (44%)
Task run 'controler-dd5' - Finished in state Completed()
Flow run 'amber-bullfrog' - Finished in state Completed()

Deuxieme lancement (modele avec fuite) :
Task run 'controler-...' - importances : statut_enquete 0.754, montant 0.285, ...
ValueError: variables non autorisees : ['statut_enquete']
Flow run '...' - Finished in state Failed(...)
```

Si le serveur Prefect du TP 7 tourne, les deux exécutions apparaissent dans son interface (Runs), l'une verte, l'autre rouge. Ajoutez cette tâche au flux `reentrainement` du TP 7, entre l'évaluation et la notification.

Étape 15 — Synthèse : le même dossier, cinq outils (15 min)

Remplissez dans le compte rendu un tableau : pour chaque outil, la prédiction pour Mme Benali, les deux causes principales, la forme de l'explication (probabilité, log-cote, règle), son caractère exact ou approché, et le niveau de code nécessaire.

RÉSULTAT ATTENDU

```
Valeurs relevees lors de la preparation :
Foret + SHAP (Python)      P 0,890   nuit +0,37, montant +0,36   probabilite, exacte
Foret + LIME (Python)      P 0,890   montant +0,45, nuit +0,21   regle, approchee
EBM (InterpretML)          P 0,984   montant +5,69, nuit +4,37   log-cote, exacte
Orange, Explain            P 0,91    montant +0,38, nuit +0,36   sans code
H2O XGBoost                P 0,975   nuit +3,38, montant +3,02   log-cote
```

Questions du compte rendu

1. Rédigez la phrase destinée à Mme Benali, puis le contrefactuel que vous lui proposeriez. Pourquoi pas « ayez 92 mois d'ancienneté » ?

2. MDI, permutation et SHAP global donnent-ils le même classement ? Expliquez la différence sur `nuit` entre MDI (0,093) et permutation (0,193).
3. LIME : quelles contributions sont stables, lesquelles changent de signe ? Quelle règle adopter avant de communiquer une explication à un client ?
4. À l'étape 7, quelle information a révélé la fuite, et pourquoi un F1 de 0,93 aurait-il dû alerter ?
5. La logistique avec log(montant) atteint 0,8593 contre 0,8636 pour la forêt. Quel modèle mettriez-vous en production pour bloquer des paiements, et pourquoi ?
6. Le modèle ne voit pas le groupe. Pourquoi les clients honnêtes du groupe B ont-ils deux fois plus de fausses alertes ? Quelle métrique d'équité choisiriez-vous, et pourquoi ?
7. Orange partage la contribution de `nuit` entre deux colonnes et H2O donne des log-cotes : comment comparer honnêtement des explications venant d'outils différents ?
8. Le garde-fou 1 de l'étape 14 repère `statut_enquete` par son nom. Si la colonne fautive s'appelait `montant_verifie`, quel garde-fou la détecterait encore ? Calculez la part de la variable en tête.
9. Quels éléments de ce TP permettraient de reconstituer la décision de Mme Benali dans six mois ?

Livrables

- Les scripts `src/` et les images de `reports/`, commités dans Git
- La phrase et le contrefactuel destinés à Mme Benali
- Le tableau de stabilité de LIME, le tableau d'équité par groupe
- La fiche modèle complétée, journalisée dans MLflow
- Partie B : le workflow Orange `.ows` et ses captures, la sortie H2O, les deux exécutions Prefect (réussie et refusée)
- Le tableau de synthèse « même dossier, cinq outils » et les réponses aux questions

Barème

Critère	Points
Explications locales : SHAP, phrase client, LIME et stabilité, contrefactuel (étapes 1 à 6)	5
Fuite démasquée et modèles interprétables mesurés (étapes 7 à 9)	4
Équité, journalisation MLflow et fiche modèle (étapes 10 et 11)	3
Orange : workflow Explain complet et lecture des résultats (étape 12)	3
H2O et garde-fou Prefect (étapes 13 et 14)	3
Synthèse comparative et réponses aux questions (étape 15)	2
Total	20